Technology Stack & Engineering Standards — Part 2: Engineering Standards, Development Workflow & Code Quality

Engineering standards within AROH exist to preserve architectural consistency, reduce cognitive overhead, and ensure that every contribution strengthens the ecosystem instead of introducing fragmentation. Standards are not established merely to enforce stylistic preferences but to create a predictable engineering environment where developers, AI coding assistants, reviewers, and future maintainers can immediately understand the structure and intent of the codebase. Every standard should reduce ambiguity, improve maintainability, and support long-term scalability. Whenever a new feature, package, or product is introduced, it should naturally conform to existing conventions instead of creating parallel approaches.

Code should always communicate intent more clearly than implementation details. Readability takes precedence over cleverness, explicitness over implicit behavior, and maintainability over unnecessary optimization. Every module should have a single, well-defined responsibility. Functions should remain focused on one logical task, classes should encapsulate cohesive behavior, and components should represent clearly identifiable pieces of the user interface. Deeply nested logic, hidden side effects, duplicated business rules, and tightly coupled dependencies should be avoided because they make future evolution increasingly expensive.

The repository should follow a predictable organizational structure where every directory has a clearly defined purpose. Products should contain only product-specific logic, while reusable functionality should be promoted into shared packages or platform modules whenever multiple products can benefit from it. Engineers should never wonder where new code belongs. If functionality is specific to one product, it remains inside that product. If it is reusable across the ecosystem, it becomes a platform capability or shared package. This distinction should remain one of the most important architectural rules throughout the lifetime of the project.

Naming conventions should emphasize clarity and consistency. Directory names should follow lowercase kebab-case to maintain compatibility across operating systems and development environments. Components, interfaces, types, and classes should use PascalCase because they represent identifiable entities within the system. Variables, functions, hooks, and utilities should use camelCase to remain consistent with TypeScript and JavaScript conventions. Constants should use UPPER_SNAKE_CASE when representing immutable values. Abbreviations should be minimized unless universally understood, and names should describe purpose rather than implementation. A developer unfamiliar with the codebase should understand the role of a file or symbol simply by reading its name.

Architectural boundaries should be enforced rigorously. Products may depend upon platform services and shared packages, but they should never depend directly on one another. Platform modules should communicate through stable contracts rather than internal implementation details. Circular dependencies are prohibited because they obscure ownership, complicate testing, and increase maintenance costs. Every dependency should move toward more stable abstractions rather than toward volatile implementation

details. The dependency graph should remain intentionally simple so that replacing one implementation never requires cascading changes throughout the ecosystem.

Business logic should remain independent of presentation logic. User interface components are responsible only for rendering information and collecting user interactions. They should not contain authorization rules, financial calculations, persistence logic, or business workflows. These responsibilities belong within platform services or application-layer use cases. This separation improves testability, enables alternative user interfaces in the future, and prevents business rules from becoming scattered throughout the presentation layer.

Error handling should be intentional and standardized across the ecosystem. Every operation capable of failure should produce meaningful, structured errors rather than generic exceptions or silent failures. Error messages should provide sufficient context for debugging while remaining safe for production environments. Internal implementation details should never be exposed directly to users. Platform services should classify errors consistently, allowing products to present appropriate user experiences without duplicating error handling logic. Logging should accompany unexpected failures so operational issues can be diagnosed without requiring users to reproduce them manually.

Validation should occur at every architectural boundary rather than only at user interfaces. Client-side validation improves usability by providing immediate feedback, but it should never be treated as authoritative. Server-side validation remains mandatory for all operations affecting business state. Shared schemas should define validation rules once so that client interfaces, platform services, documentation, automated testing, and APIs remain synchronized. Validation logic should not be duplicated across layers because duplicated validation inevitably diverges over time.

Security principles should influence every engineering decision. Sensitive operations such as authentication, authorization, wallet management, membership upgrades, administrative actions, and financial transactions must always remain server-authoritative. Client applications should never be trusted to enforce security policies independently. Every privileged operation should be validated through centralized platform services, audited appropriately, and protected against unauthorized modification. Engineering convenience should never justify compromising security architecture.

Documentation should evolve simultaneously with implementation. Every architectural decision, public API, shared package, platform service, deployment procedure, migration, and operational workflow should be documented before it is considered complete. Documentation should explain not only how something works but also why particular architectural decisions were made. Future developers and AI assistants should understand the reasoning behind the system rather than reverse-engineering intentions from implementation alone. Outdated documentation should be treated as a defect because it undermines architectural consistency.

Testing is considered an integral component of implementation rather than a separate activity performed after development. Unit tests should verify isolated logic, integration tests should validate interactions between platform services, and end-to-end tests should confirm complete user workflows across products. Shared packages should maintain their own testing strategies independent of consuming applications. Regression testing should ensure that new functionality never compromises existing capabilities. Testing should focus on protecting architectural contracts and business rules rather than merely increasing code coverage metrics.

Code reviews should evaluate architecture before implementation details. Reviewers should first determine whether a change belongs in the correct architectural layer, whether responsibilities are appropriately assigned, and whether reusable capabilities should instead become platform services. Only after architectural concerns have been addressed should implementation quality, naming, formatting, optimization, and stylistic considerations be evaluated. This review order reinforces the principle that architecture governs implementation rather than the reverse.

AI-assisted development is treated as a permanent part of the engineering workflow. AI should accelerate implementation, documentation, testing, debugging, refactoring, and architectural analysis without replacing engineering judgment. Every prompt used for AI-assisted development should reinforce platform standards, architectural boundaries, coding conventions, and documentation requirements. Generated code should be reviewed according to the same standards applied to manually written code. The objective is not merely to generate code faster but to maintain architectural integrity while increasing engineering productivity.

Continuous integration should automatically validate formatting, linting, type safety, testing, dependency integrity, and build correctness before changes are merged. Automation should prevent architectural regressions whenever possible. Continuous deployment should follow predictable release procedures with environment isolation, rollback capability, and monitoring support. Operational reliability should emerge from disciplined engineering practices rather than manual intervention.

Ultimately, engineering standards within AROH exist to create a codebase that remains understandable, scalable, secure, and maintainable regardless of how many products, developers, services, or years of development the ecosystem eventually encompasses. Every standard should reduce future complexity, strengthen architectural consistency, and contribute to the long-term objective of building a platform that compounds engineering value rather than accumulating technical debt.